Nemesis: An Exploitative Poker Simulator for Android

Clark Kirby · Project Update 4 · 10/20/2026

***Overview*—Nemesis is an Android app that shows a live low-stakes poker player which starting hands make money at their specific table. The user describes the table in plain English ("people raise light to 20, call light, can't fold top pair"), the app turns that into a parameterized opponent model, and a Monte Carlo simulator plays all 169 starting hands against it from each selected position. Results appear as a 13×13 heatmap of expected value in big blinds, with 95% confidence intervals to separate real edges from noise, and an advisor turns the numbers into a short game plan. Since Update 3, the tested Python pipeline was ported to Kotlin and wrapped in a Jetpack Compose app that follows MVVM, stores runs in Room, keeps settings in DataStore, and runs simulations in a foreground service. Claude parses descriptions and writes reports when the user supplies an API key, but every feature also works offline, so the app can be graded with no setup. The app has four tabs (Analyze, Table, History, Settings), 44 unit tests and 25 instrumented UI tests that run on every push, and is published as an installable APK.**

I. Introduction

So far, most of my time had gone into the back end: the Monte Carlo simulation that estimates how profitable each hand is against a described table. As I said in Update 3, I wanted that pipeline working before putting "a pretty coat of paint" over it. This update covers the second half, turning the simulator into a mobile app.

Standard advice and most study tools assume opponents play close to optimally, but players at a $1/$3 table usually do not. They call too much, bluff too little, and open huge. The best strategy against them depends on exactly how the table is bad, so I wanted a tool where I describe what I have seen in a sentence or two and see, position by position, which hands to open and fold. Since I would use it between sessions or on a break, a phone app made more sense than desktop Python scripts. The goal was a complete multi-screen app that runs on-device, works offline, and is tested well enough that I trust its numbers.

II. Related Work

GTO solvers such as PioSOLVER and GTO Wizard compute equilibrium strategies, which are great for fundamentals but assume the opponent also plays well, and they are expensive and take expertise to set up. Nemesis does the opposite: it assumes a fixed, flawed opponent and finds the best response. Trackers like PokerTracker and Hold'em Manager model individual opponents, but only from online hand histories. A live player has impressions, not hand histories, and Nemesis is built around turning those impressions ("the guy on my left calls everything") into numbers a simulator can use.

III. Application Design

The app is written in Kotlin with Jetpack Compose and Material 3 and follows MVVM, the pattern Google recommends for Compose [3], [4]. Each screen is a composable (the View) backed by a ViewModel that exposes state as a StateFlow; the composable redraws when the state changes and sends user actions back as function calls. I chose MVVM because Compose is built around observing state, and a ViewModel survives rotation, which matters for a screen showing a simulation in progress.

The code is split into **engine** (evaluator, opponent model, simulator), **parsing** and **advisor** (text to models, models to reports), **data** (Room, DataStore, Claude client) and **ui**. Dependencies are wired by hand in a single AppContainer instead of a framework like Hilt, which kept the project small.

A. Navigation

Type-safe Navigation Compose routes back four bottom-bar tabs (AnalyzeRoute, TableRoute, HistoryRoute, SettingsRoute) and two detail screens, ResultsRoute(runId) and ReportRoute(runId). Passing only the database id means detail screens always load from Room, so a fresh run and an old one from History behave the same.

B. Model: the opponent and the simulator

Villain describes an opponent pool with eight numbers: openRaisePct, threebetPct, callOpenPct, openSizeBb, station (reluctance to fold postflop, 0 to 1), cbetFreq, bluffFreq and stackBb, the same tendencies as the Update 3 villain mock-up. Values are clamped to the model's designed range so a bad parse cannot produce nonsense.

The main entry point is Simulator.runStats(villain, positions, nHands, openSize, nSeats, subset, seatModels, rake, seed, onProgress). For each position and hand it runs simulateHand() nHands times (600 by default) across coroutines and returns position → hand → Cell, where Cell holds the mean, standard error and sample count and computes the 95% interval and a crossesZero flag used to hatch noisy cells. seatModels gives each seat its own Villain, and seed makes runs reproducible for tests. Postflop logic matches the Python engine, verified by

porting its test suite; preflop was made more realistic (only seats behind the hero act, blinds count toward bets, results are measured against folding).

C. Controller logic: SimulationRunner

SimulationRunner lives in the AppContainer and exposes a StateFlow of a sealed SimState: Idle, Parsing, Running(progress, villainTag, source), Done(runId) and Failed(message). start(description, positions, useTableReads) parses, simulates, saves to Room and moves through those states. The AnalyzeViewModel just forwards input to start() and cancel(); the screen animates between form, progress bar and error, and navigates to the results on Done. The description and toggle live in a SavedStateHandle, so they survive rotation and process death.

D. Background work and persistence

A full run is heavy for a phone, so when one starts a foreground service shows a progress notification with Cancel, then a "Results ready" notification that opens the run [6]. Each run is saved to Room [5] as a RunEntity with the description, parsed Villain, settings, results grid, report and follow-ups; History lists them with swipe-to-delete and undo. Settings and per-seat reads are kept in DataStore, and the API key stays on the device and out of backups.

IV. LLM Integration and Offline Mode

With an Anthropic API key, Claude [7] parses the table description and seat notes into Villain parameters, writes the advisor report, and answers follow-ups, as planned in Update 3. Without a key, descriptions are parsed with keyword rules ("calls light" widens the calling range, "can't fold" raises station) and the report uses a template. In both cases the sim's average EV per hand class is turned into a query, and TF-IDF search over 12 built-in strategy principles picks the ones that ground the report. Each screen shows which path was used.

V. Results

The finished app covers everything planned in Update 3 with a different layout: the mock-ups' Villains, Ranges, Hand and Session tabs and dark theme became Analyze, Table, History and Settings with a light theme I found easier to read. The villain chat became the Analyze and Live Table screens (Fig. 1): the user describes the table, picks positions and taps Run, and can add a note for any seat to model it individually.

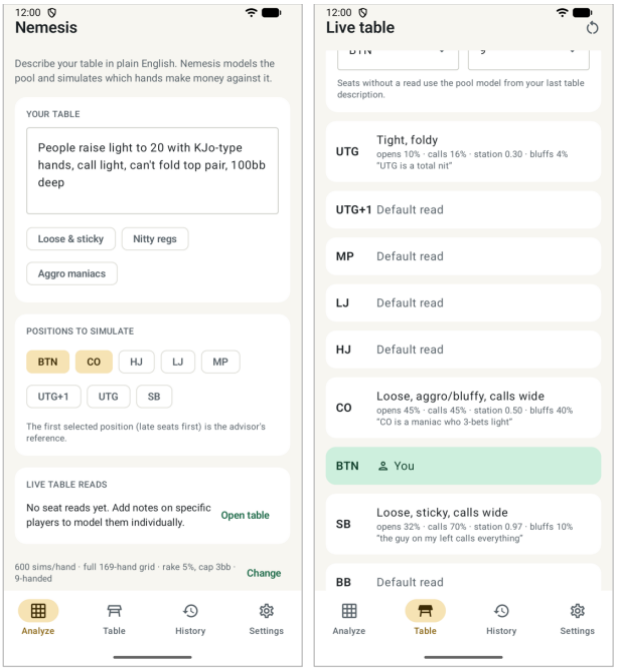


Fig. 1. Analyze screen with a table description and positions (left); Live Table with per-seat reads (right).

The heatmap sits at the center of the Results screen (Fig. 2), below the inferred opponent model. Cells run from red to green, and cells whose interval crosses zero are hatched. Tapping a cell shows its EV, interval and verdict; here KJo on the button is +4.9 bb per open against a loose, sticky table. Ranked lists of best, worst and too-close hands follow.

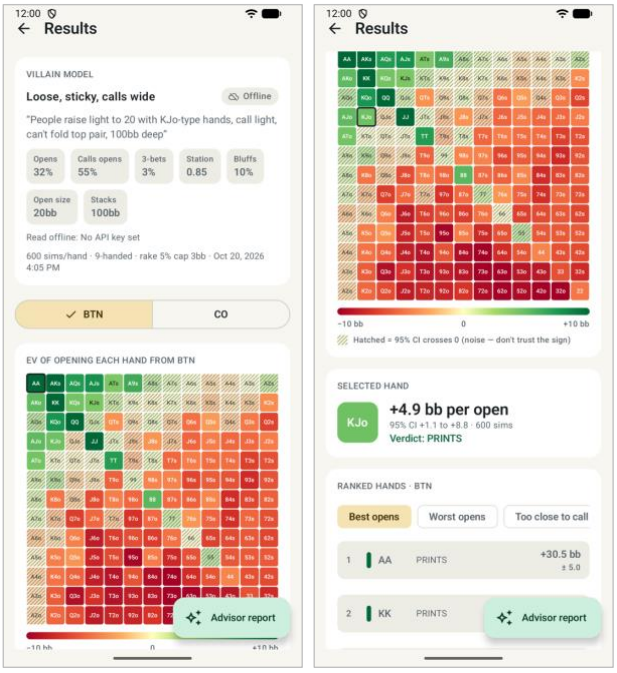


Fig. 2. Results: opponent model and button heatmap (left); a selected hand and ranked list (right).

The Advisor (Fig. 3) gives a table read, per-class results and a plan. Here it correctly says to value-bet more, stop bluffing and fold small pairs and suited connectors, the standard advice against calling stations. Reports support follow-up questions

and sharing, and every run is saved to History.

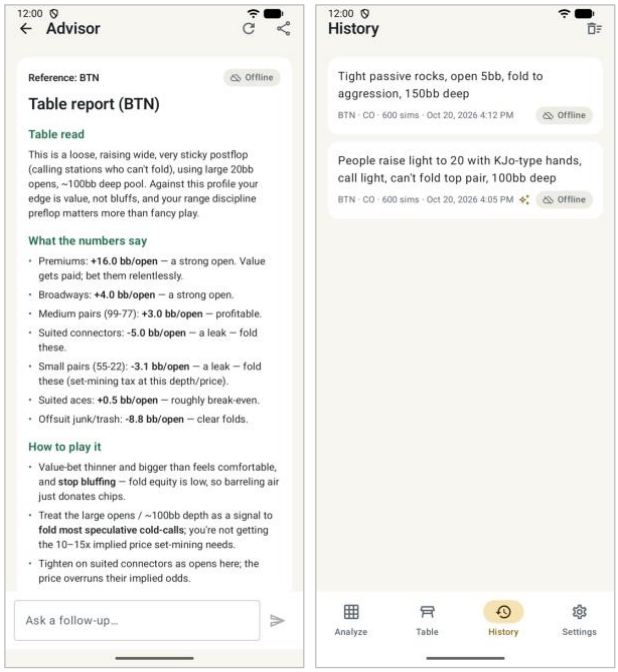


Fig. 3. Offline advisor report (left) and saved runs in History (right).

The 44 unit tests cover the engine (ported from the Python suite: hand normalization, evaluator, rake, seeded reproducibility), the parsers and the advisor. The 25 instrumented tests drive full Compose flows, heatmap tap targets, Room, DataStore and the foreground service, each with a fresh in-memory database. GitHub Actions runs both suites on an API 35 emulator on every push and builds the release APK [2].

VI. Future Work

Results are directional, not exact: villains play a fixed strategy and the hero plays one open-and-value-bet line, so the app ranks hands well but is not a precise win rate. A richer postflop model would help at the cost of speed. I also found that a villain's hand is drawn uniformly from the 169 hand classes instead of weighted by combinations, so pairs and suited hands appear more often than in a real deal. This came from the Python engine, so fixing it means updating both engines and the parity tests together.

With more time I would add hand-history import, reads that update during a session, server-side simulation for speed, and a smarter offline parser, since keyword rules miss phrasings Claude handles easily. The release APK is signed with the debug key for grading and would need a real key before the Play Store.

VII. Discussion

The UI work went much faster because the back end was already tested. Porting the Python tests first meant any disagreement with Python was a porting bug, not a modeling question. The one deliberate break from parity was preflop, where the original simplifications made results unrealistic; afterward I retuned the advisor's thresholds to match. Source, README and APK are on GitHub [2].

References

[1] C. Kirby. (2026). *Nemesis: exploitative poker EV simulator* [Online]. Available: https://github.com/ckirby04/Nemesis
[2] C. Kirby. (2026). *Nemesis (Android)* [Online]. Available: https://github.com/ckirby04/nemesis-android-application
[3] Google. *Jetpack Compose* [Online]. Available: https://developer.android.com/compose
[4] Google. *Guide to app architecture* [Online]. Available: https://developer.android.com/topic/architecture
[5] Google. *Save data in a local database using Room* [Online]. Available: https://developer.android.com/training/data-storage/room
[6] Google. *Foreground services overview* [Online]. Available: https://developer.android.com/develop/background-work/services/fgs
[7] Anthropic. *Claude API documentation* [Online]. Available: https://docs.anthropic.com
[8] I. Hendley. *treys: Python poker hand evaluation library* [Online]. Available: https://github.com/ihendley/treys